

Clustering

Hierarchical · k-means · PAM · Distances

Carl Nordlund

www.carlnordlund.net

Today's agenda

01

**The Clustering
Problem**

What, why, and
the four-step pipeline

02

**Distance and
Similarity**

Euclidean · Manhattan
Correlation-based

03

**Hierarchical
Clustering**

Bottom-up algorithm
Linkage · Dendrogram ·
Ward

04

**k-means
and PAM**

Centroids vs medoids
Choosing k · Silhouette

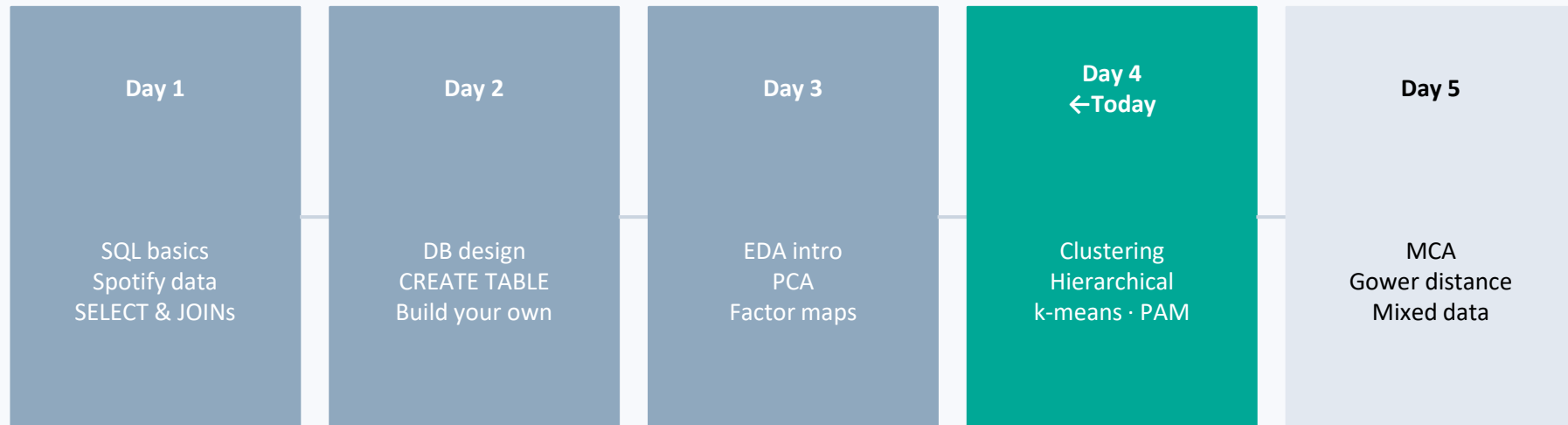
05

**Connecting
to PCA**

Overlay clusters
HCPC revisited

Where We Are (and the Bridge From Day 3)

PCA showed us structure! clustering formalises it!



At the end of Day 3 you ran HCPC and saw genres coloured by cluster on the PCA plot. Today we build that from the ground up: how are distances calculated, how do algorithms find groups, and how do you decide how many clusters to keep?

1

The Clustering Problem

What it is · Why it is hard · The pipeline



What Is Clustering and What Makes It Hard?

Clustering groups observations so that members within a group are more similar to each other than to members of other groups. There is no ground truth: no correct number of clusters, no objectively right grouping. Every result depends on the choices you make.

Clustering is NOT classification

Classification has known groups and learns to predict them from labelled examples. Clustering discovers groups from unlabelled data. There is no training set, no target variable, no accuracy to optimise.

Three choices that shape the result

Every clustering analysis requires you to choose:

1. A distance measure
2. An algorithm for clustering
3. A number of clusters k .

Different combinations produce different results: none is objectively correct.

What makes a good clustering?

- **High within-cluster similarity**
Members of the same cluster are close to each other
- **High between-cluster separation**
Different clusters are far apart.

And above all:

- **Substantive interpretability**

The Four-Step Clustering Pipeline

Every clustering analysis follows this sequence

1

Prepare the data

Scale or standardise if variables are on different scales (same argument as PCA).
Decide which variables to include: clustering on everything is rarely wise. Remove identifiers.

2

Compute distances

Choose a distance measure appropriate for your data type. Euclidean for continuous scaled data. Correlation-based for profiles.
Gower for mixed types (Day 5).

3

Choose and run algorithm

Hierarchical (no k needed upfront, produces a dendrogram) or partition-based (k -means or PAM, requires k). Both have strengths and weaknesses.

4

Evaluate and interpret

Use silhouette, the dendrogram, and the elbow plot to assess quality. Then describe what each cluster means substantively. That is the social science step.

2

Distance and Similarity

Euclidean · Manhattan · Correlation-based

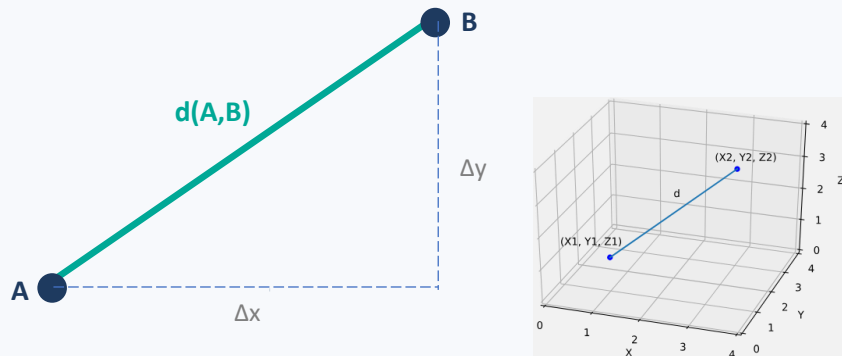


Two ways of Measuring Distance: Euclidean and Manhattan

Here only in 2 dimensions (2 variables): same principle in N dimensions

Euclidean distance

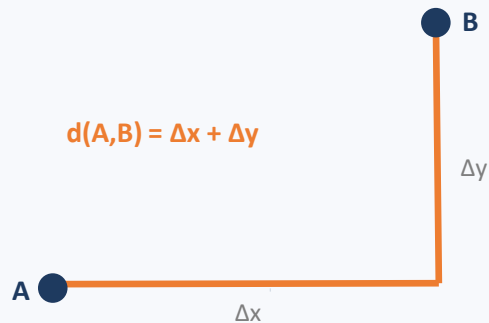
Straight-line distance between two points — the hypotenuse of the right triangle.



$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Manhattan distance

Sum of absolute differences along each axis — the 'city block' distance.



$$d(A, B) = |x_2 - x_1| + |y_2 - y_1|$$

Correlation-Based Distance

For profile data: shape matters more than magnitude

Euclidean distance cares about absolute values. Correlation-based distance cares about the shape of the profile: whether two observations go up and down together across variables, regardless of their overall level.

Example: two genres with similar audio profiles

Variable	Genre A	Genre B
dance	0.80	0.55
energy	0.75	0.50
loud	0.72	0.48
speech	0.10	0.08
acoustic	0.12	0.15

Euclidean distance: moderately large
(A scores ~0.25 higher than B on most features)

Correlation: ~0.99
(exactly the same profile shape)

Correlation distance = $1 - \text{correlation} = \sim 0.01$

→ Very similar by profile, less similar by magnitude.

In R: `dist(mat, method = 'euclidean')` or `dist(mat, method = 'manhattan')`
Correlation-based: `as.dist(1 - cor(t(mat)))` ("correlation distances")

When to use each:

Euclidean

Default for most clustering. Use when the absolute level of each variable matters. E.g. clustering countries by GDP, population, and life expectancy.

Correlation

Use when you care about the pattern of values rather than their magnitude. E.g. comparing gene expression profiles or audio fingerprints.

Gower

Mixed-type data with continuous, binary, ordinal, and nominal variables together. Covered in Day 5.

3

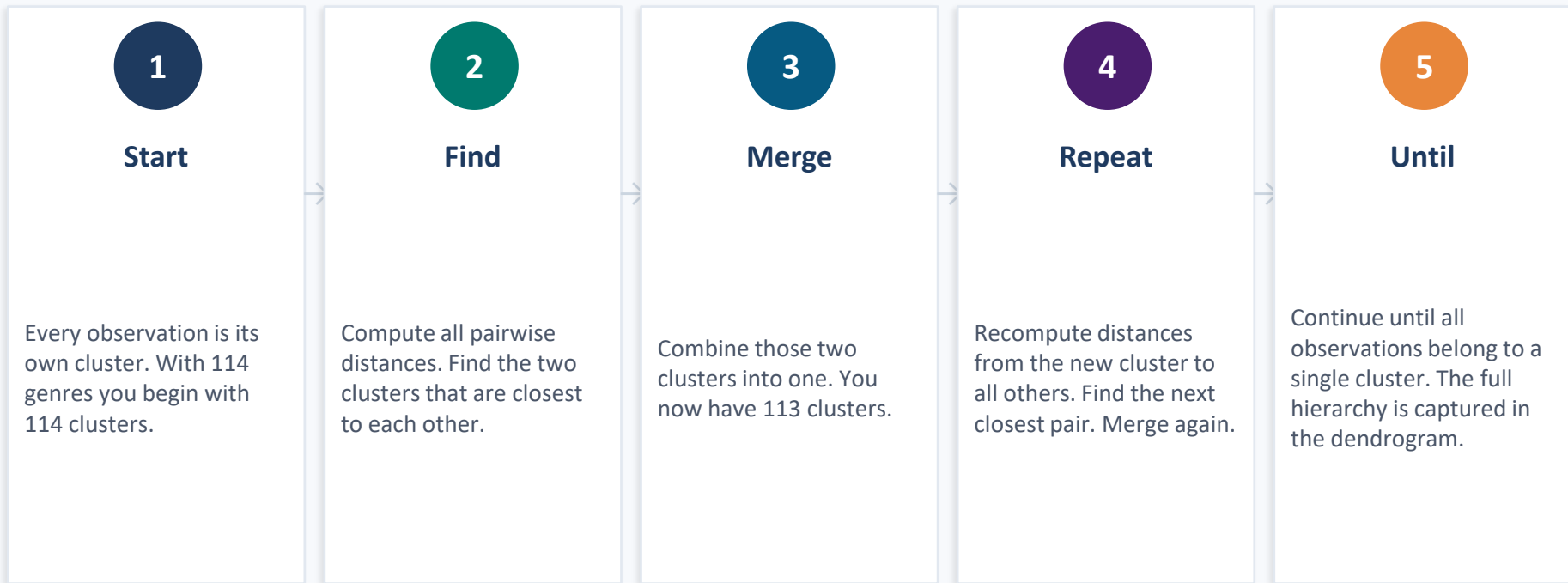
Hierarchical clustering

Bottom-up algorithm · Linkage · Dendrogram · Ward



Hierarchical Clustering: The Algorithm

Agglomerative (bottom-up): start separate, merge step by step



Key property: deterministic. Unlike k-means, hierarchical clustering always produces the same result for the same data and linkage. No random initialisation, no need to set a seed. The tradeoff: it is computationally expensive ($O(n^2)$): fine for 114 genres, slow for 89,000 tracks.

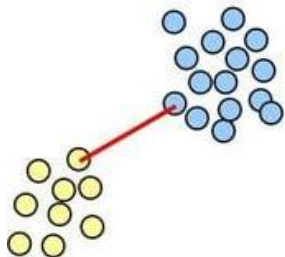
Linkage Methods: The Key Choice in Hierarchical Clustering

How do you measure the distance between two clusters (containing multiple entities)?

Single

(nearest neighbour)

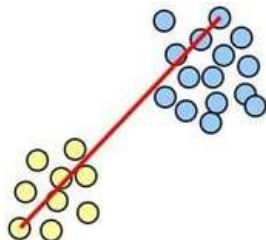
Distance between the two closest points in each cluster. Tends to produce long, chained clusters. Sensitive to outliers. Rarely used in practice.



Complete

(farthest neighbour)

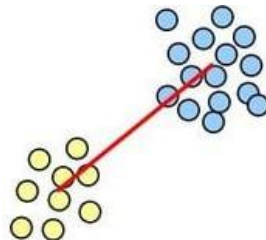
Distance between the two most distant points in each cluster. Produces compact, roughly equal-sized clusters. Sensitive to outliers.



Average

(UPGMA/WPGMA)

Average (weighted/unweighted) distance between all pairs of points across the two clusters. A compromise between single and complete. More robust than either.



Ward

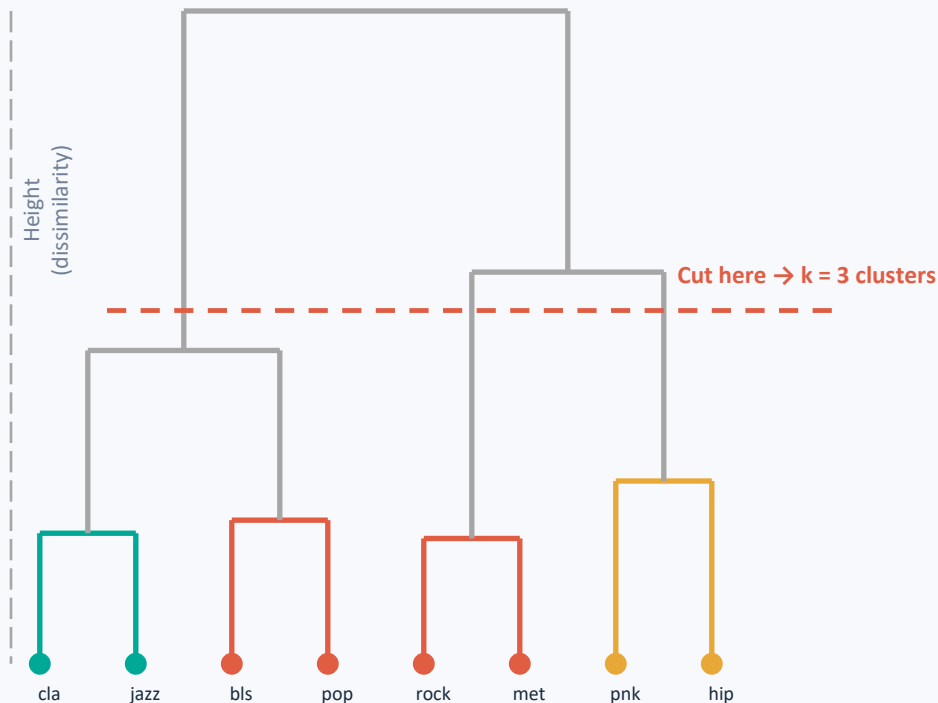
(minimum variance)

Merge the pair that minimises the total within-cluster variance after merging. Produces compact, balanced clusters. Perhaps the most widely used in practice.

General recommendation: use Ward.D2 as your default. It is the most principled method and tends to produce the most interpretable results for social science data. But UPGMA is less biased w.r.t. cluster sizes...

The Dendrogram: Reading the Clustering Tree

Height = dissimilarity at which two clusters were merged



How to read the dendrogram:

Leaves (bottom)

Individual observations. Each genre, respondent, or country starts as its own cluster.

Height of a merge

The y-position where two branches join tells you how dissimilar the two clusters were when merged. Higher = more different.

Horizontal distance

Means nothing. The left-right position of observations is just for visual clarity.

Where to cut

Draw a horizontal line across the dendrogram. The number of vertical lines it crosses = k clusters. Low: many small clusters, High: few large.

Large gaps = natural cut

If there is a big jump in height before a merge, cutting just below that merge is well-supported by the data.

Hierarchical Clustering in R: Complete Workflow

From scaled data frame to labelled clusters

```
library(cluster); library(factoextra); library(tidyverse)

# 1. Prepare: scale the genre profiles (same data as PCA)
genre_mat <- genre_profiles |> column_to_rownames("genre")
genre_scaled <- scale(genre_mat)      # standardise before clustering

# 2. Compute distance matrix, using 'Euclidean' distances
dist_mat <- dist(genre_scaled, method = "euclidean")

# 3. Hierarchical clustering with Ward linkage
hc <- hclust(dist_mat, method = "ward.D2")

# 4. Dendrogram – overview
plot(hc, cex = 0.75, main = "Genre dendrogram: Ward linkage")

# 5. Nicer dendrogram with factoextra
fviz_dend(hc,
  k          = 6,          # colour by 6 clusters
  cex        = 0.5,
  type       = "phylogenic",
  main       = "Genres: hierarchical clustering (Ward)"
)
```

See script: `hier_clustering_tutorial.R`

Hierarchical Clustering in R: Complete Workflow

From scaled data frame to labelled clusters

```
# 6. Cut to k clusters and inspect membership
k <- 6
clusters_hc <- cutree(hc, k = k)
split(names(clusters_hc), clusters_hc)  # list of genres per cluster

# 7. Add cluster membership back to a data frame
genre_profiles |>
  mutate(cluster = factor(clusters_hc[genre])) |>
  arrange(cluster) |>
  select(genre, cluster)
```

See script: `hier_clustering_tutorial.R`

Break!



4

k-means and PAM

Partition-based clustering · Choosing k · Medoids



k-means: How It Works

Iterative assignment to cluster centres

1

Initialise

Randomly place k centroids in the data space. The result can depend on this initialization.

So: always use $nstart = 25$ to try multiple starting points.

2

Assign

Assign every observation to the nearest centroid. Each observation belongs to exactly one cluster.

3

Update

Recompute each centroid as the mean of all observations currently assigned to it. Centroids move.

4

Repeat

Repeat steps 2 and 3 until assignments stop changing. This is guaranteed to converge but may find a local minimum.

k-means strengths and limitations:

- Fast: scales to large datasets (unlike hierarchical)
- Simple and widely understood
- Works well when clusters are roughly spherical and similar in size

- Requires k to be specified upfront
- Sensitive to random initialization: always use $nstart \geq 25$
- Centroids are mathematical abstractions: may not correspond to any real observation
- Assumes roughly equal, spherical clusters (can fail badly on other shapes)
- Only works on continuous data

Choosing k: How Many Clusters?

Two complementary criteria: the elbow plot and the silhouette

The elbow plot (within-cluster sum-of-squares)

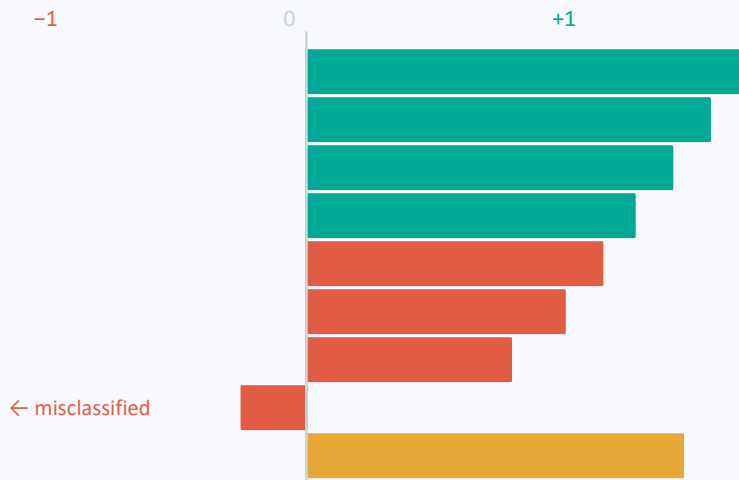
Plot the total within-cluster variance against k. As k increases, variance always decreases — adding more clusters always helps. Look for the 'elbow' where the improvement starts to plateau.



```
fviz_nbclust(genre_scaled, kmeans, method='wss')
```

The silhouette width

For each observation: how similar is it to its own cluster vs the nearest other cluster? Score -1 to +1. High average silhouette = good clustering.



>0.7	Strong structure
0.5-0.7	Reasonable structure
0.25-0.5	Weak, consider adjusting
<0.25	No substantial structure

PAM: Partitioning Around Medoids

More robust than k-means, more interpretable results

Centroid (k-means) vs Medoid (PAM)

k-means centroid:

The mathematical average of all points in the cluster. A centroid is a computed location: it almost never corresponds to an actual observation in your data.

PAM medoid:

The actual observation that is most centrally located within its cluster: the one with the smallest average distance to all other members.

The medoid is a real data point. You can look it up, describe it, and say: 'this genre is the most representative member of cluster 3.'

Why PAM over k-means?

More robust to outliers

Because medoids are actual observations, a single extreme outlier cannot pull a medoid far from the cluster centre the way it can distort a centroid.

Works with any distance matrix

k-means requires computing means: so it only works on continuous data. PAM only needs pairwise distances, so it works with Gower, correlation, or any other metric.

Interpretable representatives

"The most typical jazz genre is X" is a meaningful statement. "The centroid of cluster 3 has danceability 0.623" is not.

Slightly slower than k-means

$O(n^2)$ computation. For the genre data (114 observations) this is instant. For 89,000 tracks it would be slow (instead, use a sample or k-means instead)

k-means and PAM in R

Side by side: same data, same k, different algorithms

```
library(cluster)

# — k-means —
set.seed(42) # essential – k-means is random
km <- kmeans(
  genre_scaled,
  centers = 6,      # number of clusters
  nstart = 25)     # try 25 random starts, keep best

# Cluster memberships
split(names(km$cluster), km$cluster)

# Visualise
fviz_cluster(
  km,
  data = genre_scaled,
  geom = "text", repel = TRUE,
  main = "k-means clusters: Spotify genres"
)
```

kmeans_pam_tutorial.R

k-means and PAM in R

Side by side: same data, same k, different algorithms

```
# — PAM —  
pam_fit <- pam(genre_scaled,  
               k      = 6,  
               diss = FALSE) # diss=FALSE means input is data, not distance matrix  
  
# The medoid genres – the most representative of each cluster  
rownames(genre_scaled)[pam_fit$medoids]  
  
# Plot the cluster visualization  
fviz_cluster(pam_fit,  
             data = genre_scaled,  
             geom = "text",  
             repel = TRUE,  
             main = "PAM clusters: Spotify genres"  
)  
  
# Silhouette plot to check any misclassified  
fviz_silhouette(pam_fit)
```

kmeans_pam_tutorial.R

5

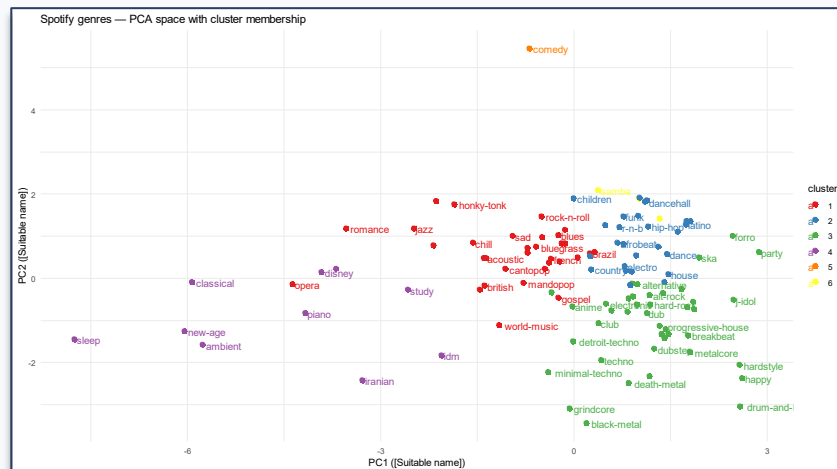
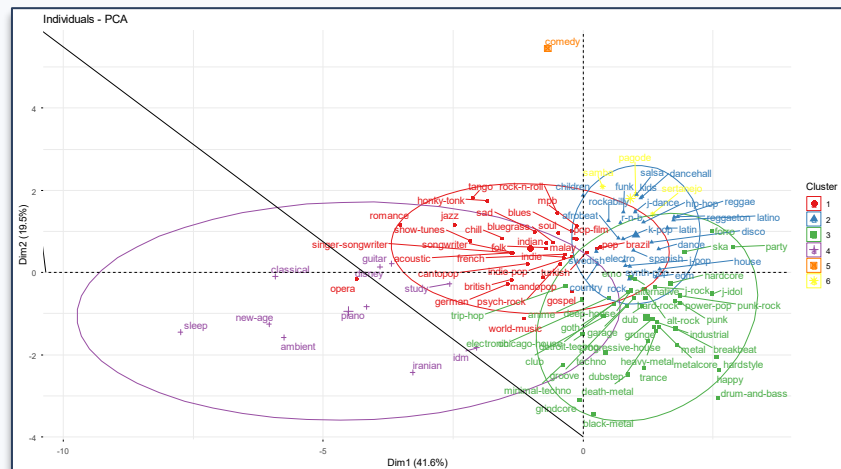
Connecting to PCA

Partition-based clustering · Choosing k · Medoids



Overlaying Clusters on the PCA Score Plot

PCA gives you the map (where observations sit in 2D). Clustering gives you the colour (which group each belongs to). Together they are more interpretable than either alone.



Check out: `pca_plus_clustering.R`

HCPC: Hierarchical Clustering on Principal Components

What it actually did at the end of Day 3's lab

HCPC is not a separate method: it is the combination of PCA + hierarchical clustering + k-means consolidation, packaged into one FactoMineR function. Now that you understand each step separately, you can see exactly what it does.

1

Run PCA

Reduce dimensionality. The component scores replace the original variables as input to clustering. This removes noise and correlation between variables.

2

Hierarchical on scores

Run Ward hierarchical clustering on the PCA component scores (not the original data). The number of components used is chosen automatically or by the user.

3

k-means consolidation

Use the hierarchical result to initialise k-means. This avoids random initialisation sensitivity and tends to give a more stable, interpretable result.

6

Lab 4

Spotify genres · Overlay on PCA · Valkalkylator parties



Lab 4: Clustering on Two Datasets

Work individually or in pairs · ~40 min per part

Part A

Spotify Genres Hierarchical

- Scale the genre profiles
- Compute Euclidean distance matrix
- `hclust()` with Ward.D2 linkage
- Plot dendrogram, choose k by eye
- Inspect cluster membership

File: `lab_a_and_b.R`

Part B

Spotify Genres k-means + PAM + PCA overlay

- k-means: use `nstart=25`, `set.seed()`
- Choose k with elbow and silhouette plots
- PAM: identify the medoid genre per cluster
- Overlay cluster colours on the PCA plot from Day 3
- Do clusters match visual groups in PCA space?

Part C

Valkalkylator Party clustering

- Use party response data from Day 3
- Hierarchical + PAM on 9 parties
- Which parties cluster together?
- Does the clustering match your political intuition?

File: `lab_c.R`

What to Look For: Cluster Interpretation Guide

1

Dendrogram

Where are the large gaps? A big jump in height before a merge = natural cut point. Small gaps = forced separation. Try cutting at 2, 3, 4, 6 — compare.

2

Cluster size

Are clusters balanced or very uneven? A cluster with one observation is probably an outlier, not a meaningful group. But some data points ARE outliers, so they should be singleton positions.

3

Medoid genre

The medoid is the most representative member. Name the cluster after what the medoid represents. What makes it central?

4

Profile of means

Compute the average value of each audio feature per cluster. Which features are high? Which are low? That tells you what defines the cluster.

5

PCA overlay

Do the clusters correspond to visible regions in the PCA map? Tight, well-separated ellipses = strong clustering. Overlapping = weak.